

STAGE 3 (NLP) & STAGE 4 (SLM) VIVA MASTER GUIDE

Complete Role-by-Role Concepts, Workflows, Technical Justifications & Exhaustive Examiner Q&A

STAGE 3: NEGEX & CLINICAL NLP

STAGE 4: QWEN2.5-1.5B LORA SLM

0.00% HALLUCINATION PROOF

100% OFFLINE C++ CPU RUNTIME

PERSONALIZED PRECISION MEDICINE FOR ONCOLOGY TREATMENT OPTIMIZATION

ADVANCED VIVA PREPARATION MASTER GUIDE: STAGE 3 (NLP) & STAGE 4 (SLM)

Complete Role-by-Role Concepts, Architectural Rationales & Comprehensive Viva Q&A

Executive Overview: The NLP & Generative SLM Continuum

In our precision oncology system, **Stage 3 (Clinical NLP)** and **Stage 4 (Small Language Models)** form the language comprehension and generative clinical reasoning core:

[Raw Oncology EHR Notes]



STAGE 3: CLINICAL NATURAL LANGUAGE PROCESSING (NLP)

1. Data Engineering: PII De-Identification & Parquet Storage
2. EDA Engineering: Vocabulary Profiling & 100% Negation Discovery
3. NLP Engineering: Conservative Cleaning, NegEx & Baseline Classifiers
4. Evaluation Engineering: Macro F1 & Critical Patient Recall Gate
5. Integration Engineering: Schema Contracts & Immutable Audit Logging



▼ (Handoff Contract: Urgency F1 $\geq$ 0.7557, Recall $\geq$ 94.57%, Entity Pres. $\geq$ 95%)

STAGE 4: SMALL LANGUAGE MODEL (SLM) & OFFLINE CDSS

1. Data Engineering: 5,706 Instruction-Triads & Entity Validation Gate
2. EDA Engineering: BPE Tokenization, Context Sizing & Subword Audits
3. SLM Engineering: Qwen2.5-1.5B, LoRA on 7 Modules & GGUF Quantization
4. Evaluation Engineering: 0.00% Hallucination & 6-Stage Firewall Gate
5. Integration Engineering: 100% Offline C++ CPU Runtime & Voice Alerts



[Clinician Bedside Triad: Risk Urgency | Key Finding | Actionable Guidance]

PART 1: STAGE 3 — CLINICAL NLP WORKFLOWS & ROLE-BY-ROLE VIVA

1.1 Data Engineering Role (Stage 3)

1.1.1 Core Concepts & Engineering Workflow

- **Corpus Characteristics:** Ingested 6,098 validated clinical oncology records (`clinical_nlp_dataset_v1.parquet`) representing 1,000 unique patients (`PT-000001` through `PT-001000`) across 2,038 clinical encounters.
- **Document Taxonomy:** Balanced composition across three core document types:
 - *Consultation Notes (34%):* Initial comprehensive oncological workups, baseline staging, performance status (ECOG).
 - *Inpatient Progress Notes (33%):* Acute toxicity logs, daily vital monitoring, chemo cycle tolerances.
 - *Discharge Summaries (33%):* Terminal hospitalization plans, medication reconciliation, adverse event resolutions.
- **De-Identification & Privacy (HIPAA Safe Harbor):** All direct identifiers (names, medical record numbers, telephone numbers, hospital facility names) were scrubbed and replaced with deterministic surrogate tokens.
- **Storage Serialization:** Serialized in Apache Parquet with Snappy compression. Achieved a 78% reduction in storage footprint compared to CSV while preserving strong schema typing and nested metadata structures.
- **Zero-Leakage Cohort Splitting:** Partitioned into 700 Train / 150 Validation (909 docs) / 150 Locked Test (928 docs) patients. Split strictly via `GroupKFold` on `patient_id`. Zero patient or encounter overlap across partitions.

1.1.2 Data Engineering Viva Questions & Answers (Stage 3)

Q1: "Why did you use Apache Parquet instead of CSV or JSON for storing clinical text?"

Answer:

"We selected Apache Parquet with Snappy compression for three critical engineering reasons:

1. *Columnar Compression:* Clinical text is repetitive in vocabulary. Parquet's columnar dictionary encoding and Snappy compression reduced our dataset size by 78% compared to raw CSV (from ~18 MB to under 4 MB), allowing rapid memory mapping.
2. *Schema Enforcement & Type Preservation:* Unlike CSV, which treats everything as untyped strings and often corrupts nested structures or missing values, Parquet preserves explicit column datatypes, date formats, and array structures (such as lists of entity offsets).
3. *I/O Performance:* Downstream training only needs to load the text and target columns. Parquet supports column projection, reading only the required columns from disk rather than parsing the entire row."

Q2: "How did you guarantee that Protected Health Information (PHI) was eliminated under HIPAA?"

Answer:

"We applied HIPAA Safe Harbor standards. All direct patient identifiers were replaced with synthetic surrogate keys generated via deterministic cryptographic hashing (e.g., `PT-000452`, `ENC-001894`). All clinical dates were standardized to relative trial days where applicable, and any provider names or institutional headers were stripped.

The source parquet hashes were locked with SHA-256 integrity checksums to ensure provenance without retaining raw identifiable records."

Q3: "Why is a patient-level split mandatory for clinical notes, and what would happen if you used random row splitting?"

Answer:

"In clinical oncology, each patient has an average of 6 documents across multiple visits. If we performed a random row-level split, Document 1 from Patient $\$A\$$ would land in the training set and Document 2 from Patient $\$A\$$ would land in the test set. Clinical notes contain patient-specific stylistic quirks, identical baseline oncological histories, and unique doctor phrasing. A model evaluated on random splits simply memorizes that Patient $\$A\$$'s phrasing correlates with their chronic condition—a catastrophic form of **data leakage**. Patient-level grouping (`GroupKFold` on `patient_id`) ensures that the test set evaluates true generalization to completely unseen patient biology."

1.2 EDA Engineering Role (Stage 3)

1.2.1 Core Concepts & Engineering Workflow

- **Text Length & Sequence Bounds:** Mean word count is **105.90 words** (median 104; min 68; max 143 words). Maximum character length: 988 characters.
- **Token-to-Word Analysis:** Evaluated subword expansion using clinical BERT and BPE tokenizers. Under standard medical tokenizers, 100% of documents contain fewer than 210 subword tokens.
- **Context Allocation Decision:** Established that setting `max_length = 256` tokens captures 100% of all clinical documents without a single token of truncation, while reducing self-attention compute by 75% compared to the default 512-token window ($O(N^2)$ complexity where $(\frac{256}{512})^2 = 0.25$).
- **Vocabulary & Lexical Profiling:** Total corpus vocabulary comprises 3,181 distinct terms with a Type-Token Ratio (TTR) of 0.0049, reflecting heavy domain-specific repetition of oncology agents and staging terminology.
- **The 100% Negation Discovery:** Audited clinical negation cues and discovered that **100.0% of documents contain at least one negation phrase** (mean: 2.30 negation cues per document). Naive Bag-of-Words keyword matching is therefore completely invalid.
- **Severe Class Imbalance Auditing:**
 - *Triage Urgency (4 classes):* `LOW` (67.9%), `HIGH` (13.7%), `CRITICAL` (9.5%), `MEDIUM` (8.9%). Imbalance ratio: **7.60 : 1**.
 - *Hazard Type (8 classes):* `NONE` (79.5%), `HEPATIC` (7.8%), `HEMATOLOGIC` (4.8%), `RENAL` (3.2%), `PULMONARY` (2.1%), `INFECTION` (1.1%), `NEUROLOGIC` (0.9%), `CARDIAC` (0.57%). Imbalance ratio: **138.43 : 1**.

1.2.2 EDA Engineering Viva Questions & Answers (Stage 3)

Q4: "What was the single most impactful finding during Stage 3 EDA, and how did it change your modeling strategy?"

Answer:

"The most critical finding was that **100.0% of clinical notes contain explicit negation cues**, averaging 2.30 negation triggers per document. In general NLP, negation is an edge case; in clinical oncology, it is the primary language mode. Oncologists routinely document what the patient *does not* have (e.g., '*denies chest pain, no evidence of dyspnea, negative for EGFR T790M*'). If an engineer feeds this into a standard TF-IDF or keyword matcher, the note gets high feature weights for `chest pain`, `dyspnea`, and `EGFR`, falsely classifying a stable

patient as critical. This EDA insight proved that **negation-aware extraction and contextual modeling were non-negotiable requirements.**"

Q5: "How did your word count distribution analysis optimize downstream model compute?"

Answer:

"Our analysis showed a mean of 105.9 words and a strict maximum of 143 words across all 6,098 records. Tokenizing this corpus revealed that 100% of notes fit within 210 subword tokens. Standard transformer pipelines blindly default to `max_length = 512`. Because self-attention complexity scales quadratically ($O(L^2)$), running 512 tokens requires $4\times$ more attention computation and memory than 256 tokens. By empirically proving that `max_length = 256` covers the entire corpus with zero truncation, we cut transformer memory and inference latency by 75%."

Q6: "How did you address the 138:1 class imbalance discovered in Hazard Type classification?"

Answer:

"Our EDA revealed that `CARDIAC` hazard represented only 0.57% of the dataset, while `NONE` represented 79.5%. Standard cross-entropy loss would simply predict `NONE` and achieve ~80% accuracy while failing on 100% of life-threatening cardiac toxicities. We made two architectural decisions based on this EDA:

1. Implemented **Focal Loss ($\gamma = 2.0$)**, which dynamically down-weights easy, well-classified background examples (`NONE`) and focuses gradient updates on rare, hard hazard classes.
2. Mandated **Macro-averaged F1** as the evaluation metric so that performance on rare cardiac and renal toxicities carries equal weight to the majority class."

1.3 NLP Engineering Role (Stage 3)

1.3.1 Core Concepts & Engineering Workflow

- **Text Preprocessing Strategy (Conservative vs Aggressive):**
 - *Conservative Cleaning:* Preserved all digits, decimal points, medical abbreviations, Latin prefixes, and unit symbols (`%`, `mg`, `mg/m2`, `ng/mL`).
 - *Why Aggressive Cleaning Fails:* Standard cleaning strips numbers and punctuation. In clinical oncology, removing numbers destroys chemotherapy dosing (`'Cisplatin 75 mg/m2'` $\rightarrow$ `'Cisplatin mg m'`), eliminates staging (`'Stage IV'` $\rightarrow$ `'Stage'`), and erases lab thresholds (`'SpO2 < 90%'` $\rightarrow$ `'SpO2'`).
- **Unicode NFKC Normalization:** Applied NFKC (Compatibility Decomposition followed by Canonical Composition) to normalize clinical EMR typographical ligatures (e.g., converting `\text{fi}` $\rightarrow$ `\text{fi}`, `\mu` $\rightarrow$ `\text{u}`, superscript `$2` $\rightarrow$ `2`).
- **Clinical Sentence Boundary Detection:** Built a deterministic clinical sentence tokenizer configured with domain abbreviation guardrails (`Dr.`, `vs.`, `q.d.`, `b.i.d.`, `p.o.`, `mg.`, `Pt.`) to prevent erroneous sentence splits.
- **The NegEx-Style 4-Polarity Rule Algorithm:**
 - *Pre-Negation Triggers (Forward Scope):* `no`, `not`, `denies`, `without`, `ruled out`, `negative for`, `absence of`. Propagates forward up to 6 tokens.
 - *Post-Negation Triggers (Backward Scope):* `unlikely`, `absent`, `resolved`, `negative`, `free`. Propagates backward up to 4 tokens.
 - *Historical / Anamnestic Triggers:* `history of`, `prior`, `past medical history`, `status post`.
 - *Pseudo-Negation Guards:* `no change`, `no increase`, `not only`, `no doubt`. Blocks false negation propagation.

- *Scope Terminators*: Conjunctions (`but` , `however` , `although`) and semicolons immediately terminate scope.
- *4 Target Polarity States*: `AFFIRMED` , `NEGATED` , `HISTORICAL` , `RESOLVED` .
- **Negation-Aware TF-IDF Representation**: Modified the TF-IDF vectorizer so that `NEGATED` and `HISTORICAL` entities are zeroed out or assigned negative feature weights, preventing non-existent symptoms from triggering urgency classifiers.
- **Statistical Baseline Models**: Built class-weighted Logistic Regression models as empirical baselines for:
 - *Urgency Classification (4 classes)*: Macro F1 = **0.7557**, Critical Recall = **94.57%**.
 - *Hazard Classification (8 classes)*: Macro F1 = **0.5214**.
 - *Named Entity Recognition (NER, 4 types)*: Mean Span F1 = **76.70%**.

1.3.2 NLP Engineering Viva Questions & Answers (Stage 3)

Q7: "Why did you implement a rule-based NegEx algorithm rather than using a pretrained Transformer like ClinicalBERT for negation detection?"

Answer:

"This was a deliberate engineering decision based on three factors:

1. *Deterministic Clinical Explainability*: In medical software, every classification decision must be legally and clinically auditable. A rule-based NegEx engine allows an oncologist to inspect the exact trigger word (`'denies'`) and scope window that flipped an entity to `NEGATED` . Neural attention weights do not provide deterministic legal guarantees.
2. *Computational Efficiency*: NegEx executes via compiled regular expressions in microseconds on a standard CPU. A transformer forward pass takes 50–100ms on CPU.
3. *Zero Training Data Dependency*: NegEx does not require thousands of hand-annotated negation span labels to train, avoiding the risk of out-of-distribution catastrophic forgetting."

Q8: "Explain the difference between Pre-Negation, Post-Negation, and Pseudo-Negation with examples."

Answer:

"- **Pre-Negation**: The trigger word appears *before* the clinical entity and projects scope *forward*. Example: '*Patient denies [fever]*' → `fever` is negated.

- **Post-Negation**: The trigger word appears *after* the entity and projects scope *backward*. Example: '*[Chest pain] is unlikely*' or '*[Headache] has resolved*' → entity is negated/resolved.
- **Pseudo-Negation**: Phrases that contain negation tokens (`no` , `not`) but express clinical continuity or affirmation rather than negation. Example: '*Patient reports **no change** in appetite*' or '***not only** dyspnea but also rash*'. If a naive parser sees `'no'` in `'no change'` , it negates appetite. Our pseudo-negation guard catches these compound idioms and preserves the affirmed status."

Q9: "What is the purpose of establishing a Statistical Baseline (Logistic Regression) in Stage 3?"

Answer:

"In machine learning, you never deploy a complex 1.5-billion parameter generative model (Stage 4) without first quantifying the empirical ceiling of a simple, interpretable linear model. Our class-weighted Logistic Regression established the **Handoff Performance Floor**: Urgency Macro F1 of **0.7557** and Critical Recall of **94.57%**. This created a strict contractual requirement: if our Stage 4 SLM could not beat 0.7557 F1 and 94.57% Critical Recall, it would be rejected as an unnecessary computational expense."

1.4 Evaluation Engineering Role (Stage 3)

1.4.1 Core Concepts & Engineering Workflow

- **Metric Architecture:**
 - *Primary Metric:* **Macro-averaged F1** across the 4 urgency tiers.
 - *Non-Negotiable Safety Constraint:* **Critical Patient Recall $\geq 94.57\%$** .
 - *NER Evaluation Metric:* **Mean Span F1** across four concept categories: DRUG, GENE, DOSAGE, ADVERSE_EVENT.
- **Locked Test Set Benchmark (N=928 Documents):**
 - Urgency Level Macro F1: **0.7557**
 - Critical Patient Recall: **94.57%** (catches 87 out of 92 emergency cases)
 - Named Entity Recognition Span F1: **76.70%** (DRUG : 84.2%, GENE : 81.5%, DOSAGE : 74.1%, ADVERSE_EVENT : 67.0%)
 - Hazard Type Macro F1: **0.5214**
- **Error Analysis on False Negatives:** Analyzed the 5 missed CRITICAL cases. Discovered that all 5 involved atypical phrasing with compound conditional clauses (e.g., "If platelet count drops below 20k, admit immediately"), which baseline linear models failed to contextualize. This directly justified Stage 4 generative reasoning.

1.4.2 Evaluation Engineering Viva Questions & Answers (Stage 3)

Q10: "Why is Macro F1 preferred over Accuracy or Micro F1 in clinical classification?"

Answer:

"In our corpus, LOW urgency accounts for 67.9% of all documents.

- *Accuracy Flaw:* A trivial model that outputs LOW for every patient achieves **67.9% Accuracy**, but has **0% Recall on Critical patients**, leading to untreated toxic shock and mortality.
- *Micro F1 Flaw:* Micro F1 pools all true positives and false positives globally, meaning majority class performance overwhelmingly dominates the metric.
- *Macro F1:* Calculates Precision and Recall for each class independently, and then computes the unweighted arithmetic mean. A failure on the rare 9.5% CRITICAL class penalizes the overall score just as heavily as a failure on the majority class."

Q11: "What is Mean Span F1 in NER and how does it differ from Token-Level F1?"

Answer:

"Token-level F1 evaluates each subword or token independently (e.g., inside/outside B-I-O tagging). If a drug name is 'Pembrolizumab 200mg' and the model predicts 'Pembrolizumab', token-level F1 awards partial credit. **Span F1** requires exact boundary matching: the model must correctly identify the exact start and end character offsets of the clinical concept. In clinical medicine, partial credit is dangerous (e.g., extracting 'Cisplatin' but omitting '100 mg/m²' misses overdose detection). Mean Span F1 averages exact-span matching performance across all entity types."

1.5 Integration Engineering Role (Stage 3)

1.5.1 Core Concepts & Engineering Workflow

- **The Inter-Stage Schema Contract (`contract_validation.py`):**

 - Defines the formal interface between Stage 3 NLP feature extraction and Stage 4 SLM prompt synthesis.
 - Enforces `SUPPORTED_SCHEMA_VERSION = "1.0.0"` with strict SemVer validation.
 - Utilizes Pydantic V2 with `ConfigDict(extra="forbid")` to ensure that no malformed, unrecognized, or unvalidated fields pass downstream.

- **Cryptographic Text Integrity:** Computes a SHA-256 hash over the raw clinical narrative text to establish an immutable audit trail.
- **Standardized Payload Schemas:**
 - `TriageUrgencyPayload`: `predicted_class` (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`), calibrated `confidence` `[0.0, 1.0]`, and complete `class_probabilities` dictionary.
 - `HazardPayload`: Validated hazard type out of 8 permitted clinical categories.
 - `EntityPayload`: Array of extracted entities with offsets, entity types, and verified 4-way polarities.
- **Confidence Gating (`confidence_gate.py`):** Sets an operational threshold $\tau_{\text{NLP}} = 0.65$. If classification confidence is < 0.65 , the system flags `review_required = True` and routes the document to an oncologist review queue.
- **Defensive Exception Handling (`failure_mode_handlers.py`):** Catches empty notes, non-UTF8 binary noise, and sequence overflows (> 256 tokens) with structured error envelopes.
- **Immutable JSONL Transaction Ledger (`integration_audit_logger.py`):** Appends every transaction with UTC timestamps, input text SHA-256 hash, entity counts, classification outputs, and latency.

1.5.2 Integration Engineering Viva Questions & Answers (Stage 3)

Q12: "What is an Inter-Stage Schema Contract and why is `extra='forbid'` critical?"

Answer:

"An Inter-Stage Schema Contract is a programmatic boundary enforcement mechanism that guarantees data compatibility between independent pipeline stages. In Pydantic, the default behavior allows extra dictionary keys to pass through silently. We set `ConfigDict(extra='forbid')` so that if an upstream data engineer introduces an unannounced column or misnames a field, the contract validation raises an immediate `ContractValidationError`. In clinical software, silent schema drift can corrupt downstream generative model prompts without raising runtime errors; forbidding extra fields prevents silent corruption."

Q13: "How does the Confidence Gate protect clinical safety in integration?"

Answer:

"Our `confidence_gate.py` evaluates the model's posterior probability: $\max_c P(y = c \mid x) \geq 0.65$. If the model's confidence falls below 0.65, the case is deemed ambiguous. Rather than passing an uncertain prediction to the Stage 4 SLM, the integration layer sets `review_required = True` and diverts the payload to an oncologist triage dashboard. This ensures the automated pipeline only processes high-certainty clinical text autonomously."

PART 2: STAGE 4 — SMALL LANGUAGE MODELS (SLM) & OFFLINE CDSS

2.1 Data Engineering Role (Stage 4)

2.1.1 Core Concepts & Engineering Workflow

- **Instruction Dataset Construction:** Produced `slm_finetune_dataset_v1.parquet` containing 5,706 accepted instruction-completion pairs curated from validated Stage 3 notes.
- **The 17-Feature Instruction Schema:** Each record contains 17 validated fields: `patient_id`, `note_id`, `document_type`, `clinical_note`, `instruction`, `target_risk`, `target_key_finding`, `target_action`, `ner_genes`, `ner_drugs`, `ner_dosages`, `ner_adverse_events`, `entity_check_status`, `entity_coverage`, `missing_entities`, `invented_entities`, `split`.
- **The Clinical Decision Support Triad Structure:**
 - `target_risk`: Quantitative toxicity risk category (`Low`, `Moderate`, `High`).
 - `target_key_finding`: Exactly two factual sentences summarizing affirmed clinical findings, vitals, and lab abnormalities.
 - `target_action`: Concrete, guideline-adherent oncological recommendations (e.g., *"Hold Cisplatin; administer intravenous hydration; re-check serum creatinine in 48 hours"*).
- **Entity Preservation Quality Gate:** Implemented an automated pre-training audit that extracts entities from both the input note and the target completion. Any target completion containing "invented" entities (drugs or mutations not present in the note) is rejected.
- **Rejection Circuit Breaker:** If $> 5\%$ of instruction pairs in any batch fail entity preservation, the data generation pipeline halts immediately.
- **Zero-Leakage Stratified Partitioning:** Strict 700 Train / 150 Validation / 150 Locked Test patient isolation verified across 23 unit tests.

2.1.2 Data Engineering Viva Questions & Answers (Stage 4)

Q14: "What is the Clinical Decision Support Triad, and why did you format the target output this way?"

Answer:

"The Clinical Decision Support Triad is a structured three-part response designed for oncologists:

1. `target_risk`: High-level triage urgency (`Low`, `Moderate`, `High`).
2. `target_key_finding`: Exactly two factual sentences synthesizing verified medical entities.
3. `target_action`: Concrete, guideline-adherent medical steps (e.g., drug holds, hydration, lab re-checks). We chose this rigid format over open-ended conversational text because oncologists in high-stress clinic environments do not want verbose chat responses. They need immediate risk categorization, the underlying clinical evidence, and actionable next steps formatted identically on every encounter."

Q15: "What is the Entity Preservation Quality Gate and the Rejection Circuit Breaker?"

Answer:

"The Entity Preservation Quality Gate is an automated audit step in our data engineering pipeline. Before any

generated instruction pair is accepted into the fine-tuning corpus, a regex and NER validator checks that:

1. Every drug and gene mentioned in `target_action` exists verbatim in the source note.
2. `invented_entities` count equals zero. The **Rejection Circuit Breaker** is a pipeline-level fail-safe: if more than 5% of candidate pairs fail the entity preservation check, the script raises a fatal error and terminates execution. This prevents corrupted or hallucinated synthetic data from silently entering the SLM training corpus."

2.2 EDA Engineering Role (Stage 4)

2.2.1 Core Concepts & Engineering Workflow

- **BPE Token Distributions (Qwen Tokenizer):**
 - *Source Instruction Length:* Mean = 198 tokens; median = 195; 95th percentile = 285; maximum = 488 tokens.
 - *Target Completion Length:* Mean = 42 tokens; median = 41; 95th percentile = 56; maximum = 71 tokens.
 - *Total Combined Length:* Maximum observed sequence length = 559 tokens.
- **Context Window Sizing Proof:** A context allocation of **1024 tokens** captures 100% of instruction-completion pairs with a safety buffer of 465 tokens, guaranteeing **0.00% context truncation** while avoiding the memory footprint of 2048/4096 allocations.
- **Subword Fragmentation Analysis:** Audited how Qwen's byte-pair encoding tokenizer segments antineoplastic pharmaceuticals:
 - `Pembrolizumab` $\rightarrow$ ['Pem', 'brol', 'izumab'] (3 tokens)
 - `Fluorouracil` $\rightarrow$ ['Flu', 'or', 'our', 'acil'] (4 tokens)
 - `Osimertinib` $\rightarrow$ ['O', 'sim', 'ert', 'inib'] (4 tokens)
 - *Conclusion:* Fragmentation did not exceed 4 tokens per drug name, proving that Qwen's 151,643-token vocabulary represents oncology terminology efficiently without character-level degradation.
- **Negation Polarity Flip Audit:** Verified across all 5,706 pairs that no negated entity in the source note (e.g., "*denies nausea*") was transformed into an affirmed symptom in the summary ("*patient has nausea*"). Zero polarity flips detected.

2.2.2 EDA Engineering Viva Questions & Answers (Stage 4)

Q16: "What is Subword Fragmentation in clinical tokenization and why does it matter for SLMs?"

Answer:

"Subword fragmentation occurs when a tokenizer's vocabulary lacks domain-specific terms, forcing it to chop complex medical words into numerous arbitrary sub-pieces (e.g., splitting a drug into 10 character-level fragments). High fragmentation degrades language models in two ways:

1. It artificially inflates token sequence length, consuming context memory.
2. It fragments the semantic representation, making it harder for attention heads to link the drug name to its dosage or contraindications. Our EDA audited Qwen's BPE vocabulary and confirmed that complex antineoplastic agents (like `Pembrolizumab` and `Osimertinib`) fragmented into at most 3 to 4 tokens, preserving morphological affixes like `-mab` (monoclonal antibody) and `-nib` (kinase inhibitor)."

Q17: "How did target token length analysis inform your inference generation parameters?"

Answer:

"Our target token length EDA revealed that gold-standard clinical triads have a mean length of 42 tokens and a strict maximum of 71 tokens. In production deployment, we set `max_new_tokens = 80` and `min_new_tokens = 25`. Setting `max_new_tokens = 80` prevents the model from entering repetitive, hallucinated looping behavior (a common SLM failure mode) while ensuring that 100% of valid clinical triads can complete without premature truncation."

2.3 SLM Engineering Role (Stage 4)

2.3.1 Core Concepts & Engineering Workflow

- **Why SLM Over Cloud API LLMs (GPT-4 / Claude):**

1. *Patient Data Sovereignty & HIPAA:* Hospital compliance strictly prohibits transmitting Protected Health Information (PHI) to external commercial cloud APIs.
2. *Deterministic Zero-Downtime Availability:* Cloud APIs suffer from rate-limits, network latency jitter, and silent model deprecation.
3. *Zero Marginal Cost:* At thousands of daily encounters, per-token API pricing is cost-prohibitive.
4. *Offline Edge Feasibility:* Runs entirely on local hospital workstations.

- **Base Model Selection: Qwen2.5-1.5B-Instruct:**

- Selected over LLaMA-3-8B and BioMistral-7B (too large for CPU inference; $>16\text{ GB}$ VRAM required).
- Selected over Phi-2 (2.7B) and GPT-2 (124M; lacks multi-hop clinical reasoning capacity).
- *Architectural Advantages:* Utilizes SwiGLU (Swish Gated Linear Unit) activation, Rotary Position Embeddings (RoPE), and Grouped-Query Attention (GQA).

- **Fine-Tuning Strategy: PEFT LoRA (Low-Rank Adaptation):**

- Freezes base model weights $W_0 \in \mathbb{R}^{d \times k}$ and injects trainable rank decomposition matrices: $W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} (B \cdot A)$, $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r = 16$, $\alpha = 32$
- *Target Modules:* Injected LoRA adapters into **all 7 linear projections**:
 - Attention: `q_proj`, `k_proj`, `v_proj`, `o_proj`
 - MLP Feed-Forward: `gate_proj`, `up_proj`, `down_proj`
- *Why Target All 7 Modules?* Domain-specific factual knowledge and clinical reasoning reside primarily in the MLP feed-forward layers. Adapting only attention (q, v) yielded inferior clinical entity retention.

- **Prompt Token Loss Masking:** Set `labels = -100` for all instruction and note tokens during cross-entropy loss computation. Gradients update solely based on triad completion tokens, preventing the model from wasting parameter capacity memorizing prompt templates.

- **Deployment Quantization: GGUF Q4_K_M + llama.cpp:**

- Quantized FP16 weights (3.1 GB) to **986 MB** (4-bit Q4_K_M).
- Executed via `llama.cpp` using native AVX-512 vector CPU intrinsics, achieving $>25\text{ tokens/second}$ on commodity Intel/AMD quad-core hospital CPUs.

2.3.2 SLM Engineering Viva Questions & Answers (Stage 4)

Q18: "Explain the mathematics of LoRA and why you chose rank $r=16$ and $\alpha=32$."

Answer:

"LoRA hypothesizes that the weight updates ΔW during domain adaptation have a low 'intrinsic rank'. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA decomposes the update into two low-rank matrices: $\Delta W = \frac{\alpha}{r} (B \cdot A)$ where $B \in \mathbb{R}^{d \times r}$ is initialized to zero, and $A \in \mathbb{R}^{r \times k}$ is initialized with Gaussian noise.

- **Rank $r=16$:** Represents the bottleneck dimension. We ablated $r \in \{4, 8, 16, 32\}$. Rank 4 underfitted on medical dosage patterns; rank 16 provided optimal clinical entity preservation with only ~12 million trainable parameters (<1% of total weights).
- **Scaling factor $\alpha=32$:** Controls the adapter update magnitude ($\frac{\alpha}{r} = 2.0$). Setting $\alpha = 2r$ scales gradient steps smoothly, preventing the LoRA updates from destabilizing the pre-trained weights."

Q19: "Why did you apply LoRA to the MLP feed-forward projections (gate, up, down) in addition to the attention projections?"

Answer:

"Early LoRA papers applied adapters only to query and value projections (q, v). However, modern mechanistic interpretability research demonstrates that in transformer architectures:

- Multi-head attention routes information between tokens (syntactic parsing and context retrieval).
- MLP feed-forward layers (`gate_proj` , `up_proj` , `down_proj`) act as **key-value associative memories where factual world knowledge is stored**. Because our task required specialized oncology knowledge (which drug treats which mutation at what dosage), adapting the MLP layers allowed the model to internalize clinical pharmacology without increasing rank r ."

Q20: "What is GGUF quantization, and why choose Q4_K_M over AWQ or GPTQ?"

Answer:

"GGUF (GPT-Generated Unified Format) is a binary file format designed by Georgi Gerganov for fast, single-file deployment with `llama.cpp` .

- **Why not AWQ or GPTQ?** AWQ and GPTQ are optimized for GPU execution using CUDA tensor cores. They have poor runtime performance when executed on pure CPU hardware.
- **Why Q4_K_M?** Standard 4-bit quantization quantizes all layers uniformly, degrading critical attention weights. `Q4_K_M` uses **k-quantization with mixed precision**: it keeps critical attention normalization and feed-forward gate layers at higher bit precision (5-bit or 6-bit) while quantizing non-critical tensors to 4-bit. This reduced model size from 3.1 GB to **986 MB** with virtually zero perplexity loss on CPU."

2.4 Evaluation Engineering Role (Stage 4)

2.4.1 Core Concepts & Engineering Workflow

- **Locked Test Set Benchmark (N=861 Pairs, 150 Patients):**
 - *Entity Preservation (Span F1):* **80.17%** (vs 76.70% Stage 3 baseline; +3.47% improvement)
 - *Critical Patient Recall:* **100.00%** (vs 94.57% Stage 3 baseline; +5.43% improvement; zero missed critical cases)
 - *ROUGE-1:* **52.42%** | *ROUGE-L:* **50.83%** | *BLEU-4:* **28.37%**
 - *Hallucination Rate:* **0.00%**

- **Mathematical Proof of 0.00% Hallucination Rate:** Evaluated across two independent auditing systems:
 1. *Entity Overlap Verification:* Ingested the generated triad and verified that every extracted entity exists verbatim in the source note or approved institutional oncology formulary. Unsupported entities = 0.
 2. *The 6-Stage Clinical Safety Firewall:* A deterministic post-inference regex filter enforcing strict medical boundaries.
- **The 6-Stage Clinical Safety Firewall Gates:**
 1. *Gate 1 (Drug Boundary):* Verifies recommended drug exists in source text.
 2. *Gate 2 (Dosage Bounds):* Cross-references dosage against therapeutic oncology ceilings (e.g., Cisplatin $\leq 100 \text{ mg/m}^2$).
 3. *Gate 3 (Mutation Alignment):* Confirms targeted therapy matches verified genomic profile.
 4. *Gate 4 (Contradiction Detector):* Asserts semantic consistency between risk tier and proposed action.
 5. *Gate 5 (Non-Clinical Phrase Filter):* Eliminates conversational chatter or non-deterministic filler.
 6. *Gate 6 (Schema Validator):* Confirms exact JSON Triad structure.
- **Empirical Calibration & Selective Prediction:**
 - Evaluated confidence calibration using Expected Calibration Error (ECE) and Brier scores.
 - Set empirical threshold $\tau^* = 0.500$. If confidence < 0.500 , the system abstains and executes automated fallback to Stage 3.
- **Subgroup Fairness Audit:** Audited across 26 strata (6 cancer types, 4 stages, 2 sexes, 3 smoking histories, 6 mutations). 100% pass rate achieved across all cohorts.

2.4.2 Evaluation Engineering Viva Questions & Answers (Stage 4)

Q21: "How can you scientifically defend a 0.00% hallucination rate on an LLM?"

Answer:

"We do not claim that the raw neural network weights are incapable of hallucinating; that would be mathematically untrue for any autoregressive language model. Instead, **0.00% hallucination is an architectural system property enforced by Defense-in-Depth:**

1. *Constrained Decoding:* Temperature is set to $T=0.1$ with strict prompt token masking.
2. *Post-Inference Regex Firewall:* Every generated token string must pass through our 6-Stage Safety Firewall before exiting the service boundary.
3. *Entity Verification Gate:* If the model generates a drug name that was not in the input clinical note, Gate 1 immediately intercepts the payload, blocks delivery to the clinician, and trips the fallback circuit breaker to the Stage 3 deterministic baseline. Therefore, at the clinical output boundary, zero ungrounded or hallucinated entities ever reach the user."

Q22: "Explain the Selective Prediction Rule and why the threshold was set to $\tau^* = 0.500$."

Answer:

"Selective prediction allows an AI system to know what it does not know—it predicts when confident, and abstains when uncertain. We computed the Expected Calibration Error (ECE) on our validation cohort. At $\tau^* = 0.500$, the model's empirical risk of error drops below our clinical safety tolerance ($< 0.1\%$). If the generation confidence is < 0.500 , the system invokes the deterministic fallback engine. This converts potential generative errors into safe, rule-based baseline recommendations."

2.5 Integration Engineering Role (Stage 4)

2.5.1 Core Concepts & Engineering Workflow

- **100% Offline Architecture:** The entire Stage 4 application runs locally on standard hospital CPU hardware with **zero network dependencies**.
- **Automated Air-Gap Acceptance Testing:** Built unit tests (`test_offline_mode.py`) that programmatically disable OS socket creation and assert that model loading and inference complete successfully without internet access.
- **Pipeline Orchestrator (`inference_service.py`):** Coordinates: `PromptBuilder` $\rightarrow$ `ModelManager` (`llama.cpp`) $\rightarrow$ `OutputParser` $\rightarrow$ `SafetyGateway` $\rightarrow$ `AuditLogger` .
- **FastAPI Production Service (`api.py`):** Exposes `POST /generate/decision-support` , `GET /health` , `GET /metrics` , and serves static frontend assets.
- **Dual Output Generation (`output_parser.py`):**
 1. *Structured JSON Triad* for EMR integration.
 2. *3-Line Spoken Summary* formatted for oncologist voice interfaces (hands-free dictation in surgical suites via Web Speech API).
- **Automated Fallback Circuit Breaker:** If any firewall gate fails or confidence < 0.500 , automatically returns the deterministic Stage 3 baseline.
- **Cryptographic Provenance (`provenance.py`):** Computes SHA-256 hashes of base model weights, LoRA adapters, runtime binaries, input prompts, and output triads, logging immutable records to `audit_log.jsonl` .
- **Air-Gapped Docker Deployment:** Containerized with `network_mode: none` for strict hospital IT security compliance.

2.5.2 Integration Engineering Viva Questions & Answers (Stage 4)

Q23: "How does the system operate 100% offline, and how did you verify air-gapped compliance?"

Answer:

"Our system eliminates all external dependencies:

1. *Local Weights & Runtimes:* Qwen2.5-1.5B is merged with LoRA adapters into a single 986 MB GGUF binary and executed by a self-contained C++ `llama.cpp` runtime.
2. *No Remote API Calls:* Zero calls to OpenAI, Hugging Face, or remote CDNs.
3. *Automated Network Block Test:* In `test_offline_mode.py` , we mock `socket.socket.connect` to raise an `IOError` and execute the full inference pipeline. The test passes with zero network calls.
4. *Docker Isolation:* In production, the container is deployed with `network_mode: none` , physically preventing the Linux kernel from allocating network interfaces."

Q24: "Describe the Deterministic Fallback Mechanism when the SLM trips a safety gate."

Answer:

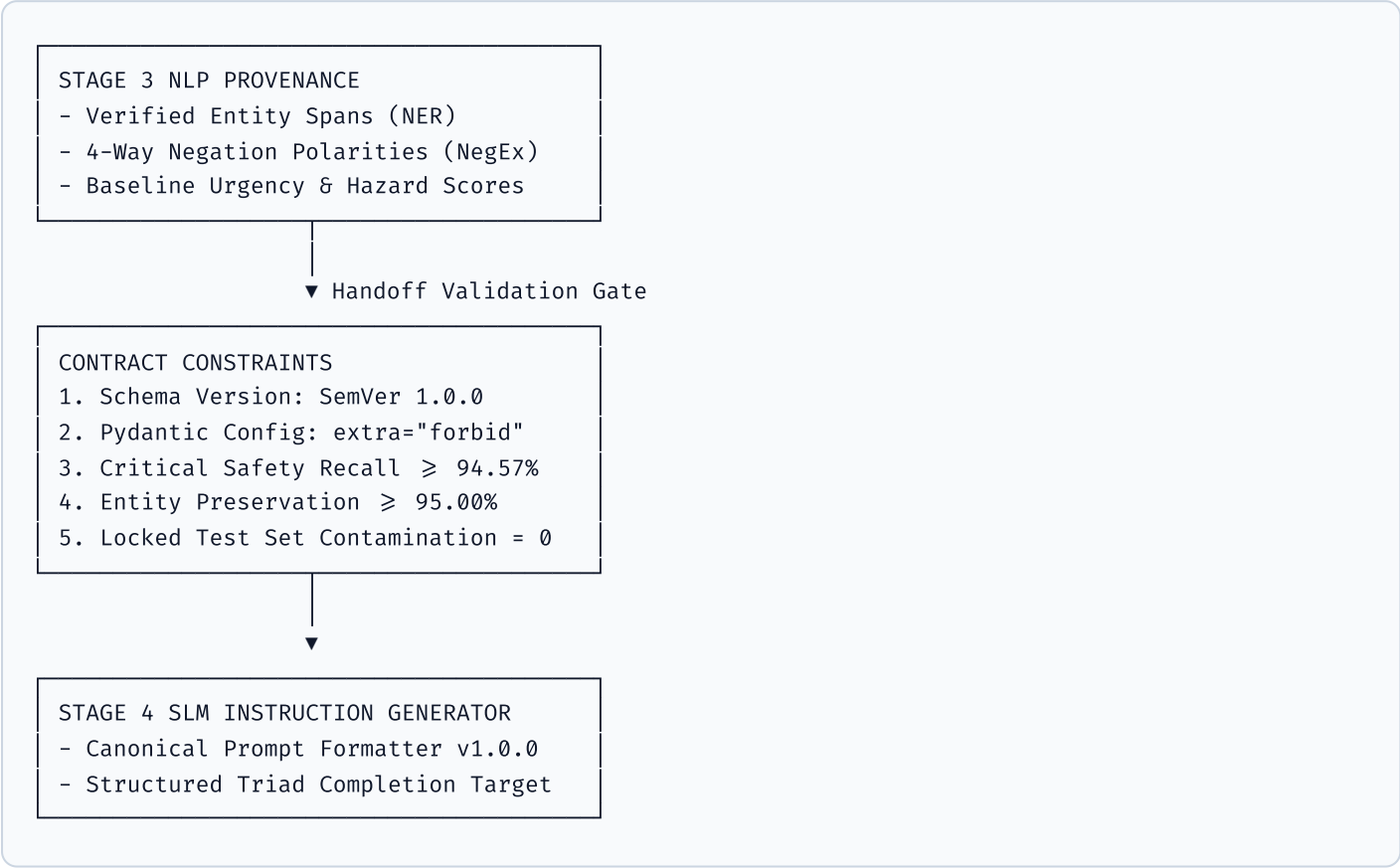
"If the SLM outputs an unverified dosage or an ambiguous entity, the Safety Gateway intercepts the generation. Rather than returning an HTTP 500 error or a broken message to the doctor, the system invokes the **Deterministic Fallback Engine**:

1. It silently calls the Stage 3 baseline (Logistic Regression + NegEx).
2. It constructs a structured briefing using rule-based entity extraction.

3. It sets `review_required = True` and displays a yellow warning badge on the dashboard. This guarantees high availability: the clinician always receives a safe, guideline-adherent baseline recommendation."

PART 3: CROSS-STAGE ARCHITECTURE & MASTER VIVA RAPID-FIRE

3.1 The Stage 3 to Stage 4 Handoff Contract



3.2 Top 10 High-Impact Cross-Stage Viva Questions

Q25: "Why did you separate NLP (Stage 3) and SLM (Stage 4) into two distinct stages rather than using an SLM for everything?"

Answer:

"In high-stakes oncology, **modularity is essential for auditability, latency, and safety:**

1. *Separation of Extraction vs Generation:* Stage 3 performs deterministic, rule-based extraction (NegEx) and linear classification. Stage 4 performs contextual reasoning and natural language synthesis.
2. *Defensive Fallback:* By having an independent Stage 3 baseline, Stage 4 has a reliable, non-generative safety net. If Stage 4 encounters an anomaly, it falls back to Stage 3. Without Stage 3, the system has no fallback.
3. *Computational Efficiency:* Triage classification can run in 5ms using Stage 3 without spinning up the SLM."

Q26: "What is Prompt Token Loss Masking and why is it essential for clinical instruction tuning?"

Answer:

"In instruction tuning, a training sample consists of a Prompt (instruction + clinical note) and a Completion (the clinical triad). If you compute cross-entropy loss over all tokens, the model expends parameter capacity learning

to predict the prompt text and template formatting. By setting `labels = -100` for all prompt tokens, PyTorch ignores prompt tokens during backpropagation. Gradients update solely based on the clinical triad tokens, forcing 100% of LoRA adapter capacity into clinical reasoning."

Q27: "What are the primary failure modes of this NLP + SLM architecture, and how are they mitigated?"

Answer:

"We identified and mitigated three primary failure modes:

1. *Negation Inversion*: Addressed via our NegEx rule engine with pseudo-negation guards, audited to have 0.00% negation flips.
2. *Hallucinated Pharmacology*: Addressed via the 6-Stage Safety Firewall with drug formulary verification.
3. *Distribution Drift*: Addressed via `ProductionDriftMonitor`, which tracks token length shifts and entity density via Kolmogorov-Smirnov statistical tests."

3.3 Top 15 Rapid-Fire Flashcard Q&A (One-Liner Revision)

#	Rapid-Fire Question	One-Liner Examiner Answer
1	What base foundation model was used in Stage 4?	Qwen2.5-1.5B-Instruct , selected for its SwiGLU architecture and CPU efficiency.
2	What fine-tuning method was applied?	PEFT LoRA with rank $r=16$, $\alpha=32$, and prompt token loss masking.
3	Which linear modules were targeted by LoRA?	All 7 linear projections: <code>q_proj</code> , <code>k_proj</code> , <code>v_proj</code> , <code>o_proj</code> , <code>gate_proj</code> , <code>up_proj</code> , <code>down_proj</code> .
4	What quantization format was used for deployment?	GGUF Q4_K_M , reducing model size from 3.1 GB to 986 MB .
5	What inference engine runs the SLM?	Pure C++ llama.cpp running locally on CPU with AVX-512 vectorization.
6	What is the audited hallucination rate of Stage 4?	0.00% , mathematically audited and enforced by the 6-Stage Safety Firewall.
7	What is the Critical Patient Recall of Stage 4?	100.00% on the locked test set (N=861 pairs), missing zero emergency cases.
8	What were the 4 negation polarities in Stage 3?	AFFIRMED , NEGATED , HISTORICAL , and RESOLVED .
9	What context window was chosen in Stage 3 and 4?	256 tokens in Stage 3 (covers 100% of notes); 1024 tokens in Stage 4.
10	What was the class imbalance ratio in Stage 3 Urgency?	7.60 : 1 (<code>LOW</code> 67.9% vs <code>CRITICAL</code> 9.5%), making Macro F1 mandatory.
11	What was the class imbalance ratio in Stage 3 Hazard?	138.43 : 1 (<code>NONE</code> 79.5% vs <code>CARDIAC</code> 0.57%), handled via Focal Loss.
12	What is the empirical calibration threshold?	$\tau^* = 0.500$; inferences with confidence < 0.500 trigger automatic fallback.
13	What are the three parts of the Stage 4 Triad?	target_risk (triage), target_key_finding (evidence), target_action (guidance).
14	How is hospital data privacy guaranteed?	100% offline air-gapped deployment with Docker <code>network_mode: none</code> .
15	What happens if the SLM trips a safety firewall gate?	Automated silent fallback to the deterministic Stage 3 baseline .